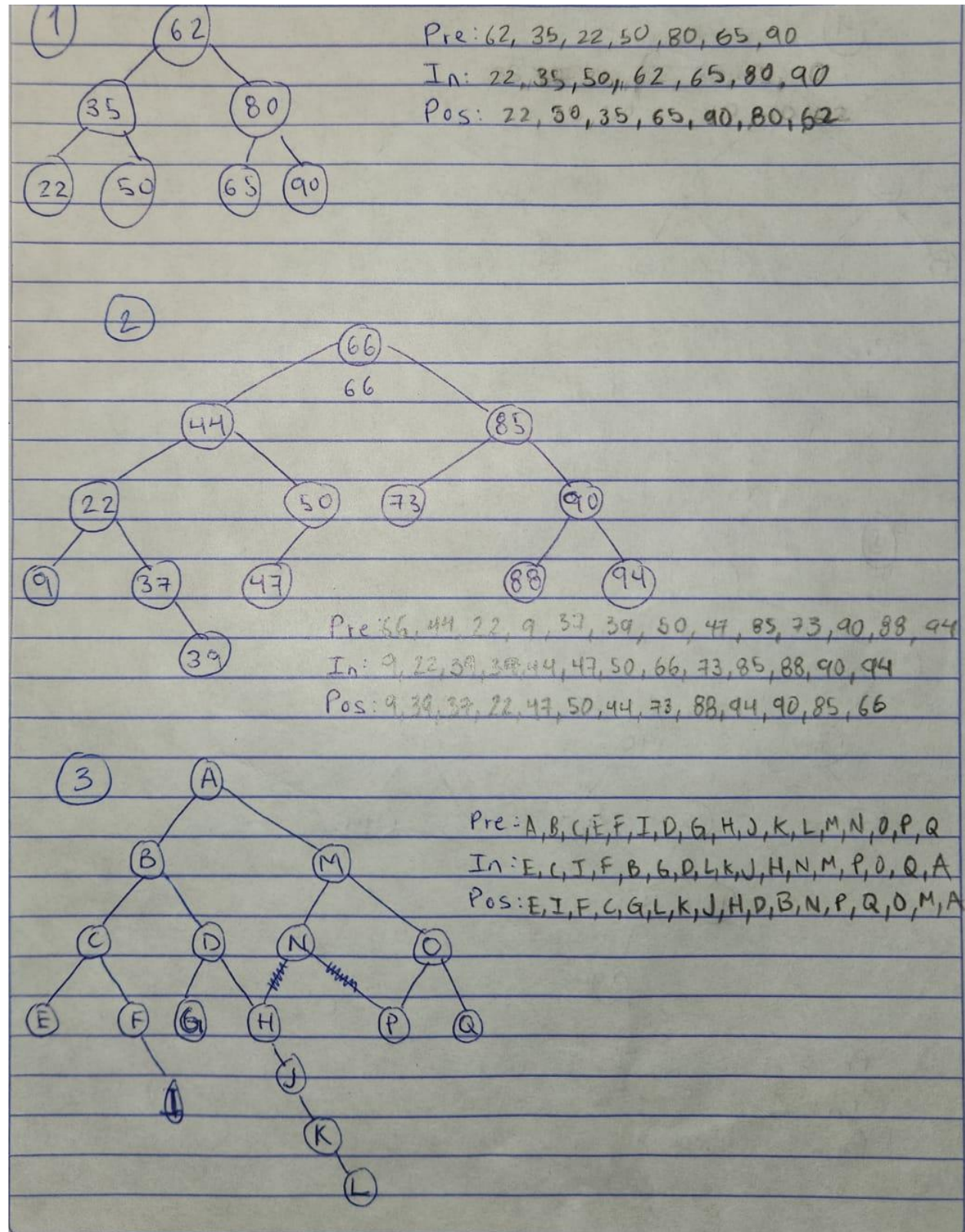


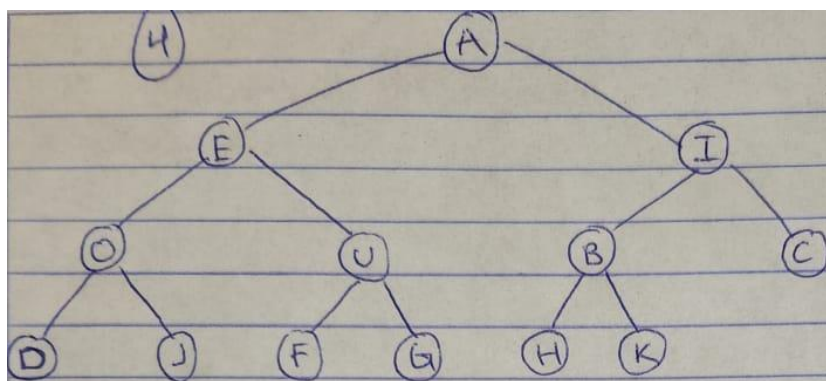


Universidad Tecmilenio
Actividad 4
Estructura de Datos
Canek Lugo Soto
2967116

Actividad 4

Recorrido de arboles

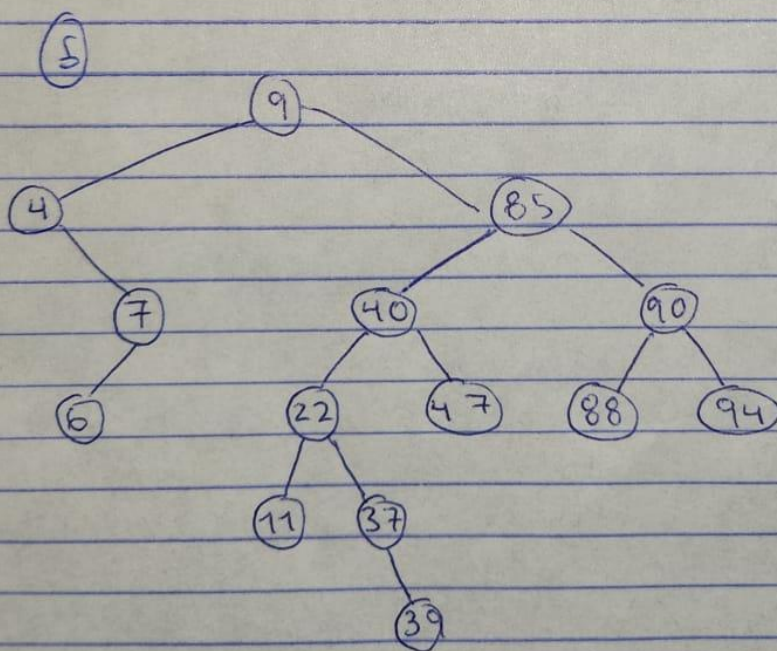




Pre: A, E, O, D, J, U, F, G, I, B, H, K, C

In: D, O, J, F, U, G, E, A, H, B, K, I, C

Pos: D, J, O, F, G, U, E, H, K, B, C, I, A



Pre: 9, 4, 7, 6, 85, 40, 22, 11, 37, 39, 47, 90, 88, 94

In: 6, 7, 4, 9, 11, 22, 37, 39, 40, 47, 85, 88, 90, 94

Pos: 6, 7, 4, 11, 39, 37, 22, 47, 40, 88, 94, 90, 85, 9

Implementación del árbol binario

```
1  package Actividad_4;
2
3  // 1. Clase Nodo
4  class Nodo {
5      int id;
6      String nombre;
7      Nodo izquierda, derecha;
8
9      public Nodo(int id, String nombre) {
10         this.id = id;
11         this.nombre = nombre;
12         izquierda = derecha = null;
13     }
14 }
15
```

En esta parte se construye la estructura básica del árbol binario. Se define la clase `Nodo`, que almacena un identificador, un nombre y referencias a los hijos izquierdo y derecho. Luego, la clase `ArbolBinario` implementa los métodos esenciales: inserción, búsqueda, eliminación y recorridos (inorden, preorden y postorden). Esta base es fundamental para manejar datos de manera jerárquica y ordenada.

Realización de operaciones

```
16 // 2. Clase ArbolBinario
17 class ArbolBinario {
18     Nodo raiz;
19
20     public ArbolBinario() {
21         raiz = null;
22     }
23
24     // Insertar un nodo en el árbol
25     public void insertar(int id, String nombre) {
26         raiz = insertarRec(raiz, id, nombre);
27     }
28
29     private Nodo insertarRec(Nodo raiz, int id, String nombre) {
30         if (raiz == null) {
31             raiz = new Nodo(id, nombre);
32             return raiz;
33         }
34         if (id < raiz.id) {
35             raiz.izquierda = insertarRec(raiz.izquierda, id, nombre);
36         } else if (id > raiz.id) {
37             raiz.derecha = insertarRec(raiz.derecha, id, nombre);
38         }
39         return raiz;
40     }
41
42     // Buscar un nodo por ID
43     public Nodo buscar(int id) {
44         return buscarRec(raiz, id);
45     }
46 }
```

```

47     private Nodo buscarRec(Nodo raiz, int id) {
48         if (raiz == null || raiz.id == id)
49             return raiz;
50         if (id < raiz.id)
51             return buscarRec(raiz.izquierda, id);
52         return buscarRec(raiz.derecha, id);
53     }
54
55     // Eliminar un nodo
56     public void eliminar(int id) {
57         raiz = eliminarRec(raiz, id);
58     }
59

```

```

60     private Nodo eliminarRec(Nodo raiz, int id) {
61         if (raiz == null) return raiz;
62         if (id < raiz.id) {
63             raiz.izquierda = eliminarRec(raiz.izquierda, id);
64         } else if (id > raiz.id) {
65             raiz.derecha = eliminarRec(raiz.derecha, id);
66         } else {
67             // Caso 1: sin hijos
68             if (raiz.izquierda == null && raiz.derecha == null) {
69                 return null;
70             }
71             // Caso 2: un hijo
72             else if (raiz.izquierda == null) {
73                 return raiz.derecha;
74             } else if (raiz.derecha == null) {
75                 return raiz.izquierda;
76             }
77             // Caso 3: dos hijos
78             raiz.id = valorMinimo(raiz.derecha);
79             raiz.derecha = eliminarRec(raiz.derecha, raiz.id);
80         }
81         return raiz;
82     }
83

```

```

84     private int valorMinimo(Nodo raiz) {
85         int minv = raiz.id;
86         while (raiz.izquierda != null) {
87             minv = raiz.izquierda.id;
88             raiz = raiz.izquierda;
89         }
90         return minv;
91     }
92
93     // Recorridos
94     public void inorden(Nodo raiz) {
95         if (raiz != null) {
96             inorden(raiz.izquierda);
97             System.out.print(raiz.id + " ");
98             inorden(raiz.derecha);
99         }
100     }
101
102     public void preorden(Nodo raiz) {
103         if (raiz != null) {
104             System.out.print(raiz.id + " ");
105             preorden(raiz.izquierda);
106             preorden(raiz.derecha);
107         }
108     }
109
110     public void postorden(Nodo raiz) {
111         if (raiz != null) {
112             postorden(raiz.izquierda);
113             postorden(raiz.derecha);
114             System.out.print(raiz.id + " ");
115         }
116     }
117 }
118

```

Aquí se aplican las operaciones sobre el árbol binario. Se insertan nodos que representan empleados, se realizan búsquedas para localizar elementos específicos y se eliminan nodos para demostrar cómo se reorganiza el árbol. También se ejecutan los recorridos clásicos para mostrar el orden de los elementos. Esta parte valida el funcionamiento de las operaciones básicas sobre la estructura.

Caso práctico: Gestión de empleados

```
119 // 3. Clase principal
120 public class Main {
    Run main | Debug main | Run | Debug
121     public static void main(String[] args) {
122         ArbolBinario arbol = new ArbolBinario();
123
124         // Lista de empleados
125         String[] empleados = {"Ana", "Canek", "Carlos", "Maria", "Jose",
126             | "Juan", "Pedro", "Laura", "Miguel", "Sofia"
127         };
128         int[] ids = {45, 20, 65, 10, 30, 50, 70, 25, 35, 60};
129
130         // Insertar empleados en el árbol
131         for (int i = 0; i < empleados.length; i++) {
132             | arbol.insertar(ids[i], empleados[i]);
133         }
134
135         // Mostrar recorridos
136         System.out.println(x:"Recorrido Inorden:");
137         arbol.inorden(arbol.raiz);
138         System.out.println(x:"\nRecorrido Preorden:");
139         arbol.preorden(arbol.raiz);
140         System.out.println(x:"\nRecorrido Postorden:");
141         arbol.postorden(arbol.raiz);
142
143         // Buscar empleados
144         int buscarId = 30;
145         Nodo resultado = arbol.buscar(buscarId);
146         if (resultado != null) {
147             | System.out.println("\nEmpleado encontrado: " + resultado.nombre + " con ID " + resultado.id);
148         } else {
149             | System.out.println("\nEmpleado con ID " + buscarId + " no encontrado.");
150         }
151
152         // Eliminar un empleado
153         int eliminarId = 20;
154         System.out.println("\nEliminando empleado con ID " + eliminarId);
155         arbol.eliminar(eliminarId);
156
157         // Mostrar recorrido después de eliminar
158         System.out.println(x:"Recorrido Inorden después de eliminar:");
159         arbol.inorden(arbol.raiz);
160     }
161 }
162
```

En esta última sección, se implementa un caso práctico con diez empleados, cada uno con un identificador único. Se insertan en el árbol binario y se realizan búsquedas por ID para demostrar su utilidad. Además, se elimina un empleado y se muestra la nueva organización del árbol. Este caso simula cómo gestionar una base de datos de empleados de manera más eficiente y estructurada.

Hay que recordar que mientras que la búsqueda lineal requiere revisar cada registro, el árbol reduce significativamente las comparaciones gracias a su estructura jerárquica. Esto permite encontrar, insertar o eliminar empleados de manera más rápida y ordenada, optimizando recursos y mejorando el rendimiento en sistemas con gran cantidad de datos.